

When Reinforcement Learning Stops Improving: Diagnosing State Aliasing in a Simple Q-Learning Experiment

1. The delivery problem

A delivery agent repeatedly faces the same basic problem: starting from one location, it must first reach a pickup point and then deliver the order somewhere else. At every step, it must decide which direction to take. Because each move affects the options available next, the problem is sequential and should be evaluated by the total cost of the route rather than one decision at a time.

Two features make the task less trivial than it first appears. First, it has two stages: the courier cannot move directly to the delivery point but must reach the pickup location first. Second, the city grid contains an obstacle that does not terminate the episode but penalizes the agent for crossing it.

The engineering question is therefore: how can an autonomous agent discover a policy that completes this two-stage pickup-and-delivery task at minimum total cost, without being given the city rules or shown an example route?

The goal is to let the agent discover a good route without explicitly programming the correct behavior. In reinforcement learning, that guidance comes from the environment, which provides feedback after each action.

2. Turning it into a two-stage MDP (Markov Decision Process)

To make the problem usable by an algorithm, we first formalize it as a Markov Decision Process (MDP).

The city is represented as an $n \times n$ grid. A bicycle starts at the center; one cell contains the pickup point P1, another the delivery point D1, and a third an obstacle. In phase 0 the bike must reach P1. Reaching it switches the environment to phase 1, in which the agent must reach D1. At that point, the episode terminates.

The main configuration studied is the 3×3 grid, small enough for every value to be checked by hand. The obstacle is modeled as a penalty tile rather than a wall: the bike may ride over it but at a higher cost. This is a modeling decision, and it matters because it is the difference between "this street is closed" and "this street is slow".

Table 1. The delivery task formalized as a Markov Decision Process (MDP)

Element	Definition
State	A state is a tuple: $s = (\text{row}, \text{column}, \text{phase})$, where $\text{phase} \in \{0, 1\}$. For a 3×3 grid: $ S = 3^2 \times 2 = 18$ states.
Actions	Four one-cell movements: $A = \{\text{up}, \text{down}, \text{left}, \text{right}\}$, with $ A = 4$. A move beyond the grid boundary keeps the agent in the same position.
Transition	Deterministic: $T(s, a, s') = 1$ for the resulting next state
Reward	-1 per step; -5 for entering the obstacle; +10 for reaching P1 in phase 0; +20 for reaching D1 in phase 1.
Terminal	Reaching D1 in phase 1 ends the episode.
Horizon	Indefinite; the episode ends when the terminal state is reached.

3. Why we deliberately used a tiny grid

We deliberately used a 3×3 grid because its 18 states are small enough to inspect and calculate by hand. That made every program output a testable prediction. If the algorithm produced a number, we could not derive ourselves, the problem was not the number; it meant there was still something we did not understand.

With that principle in mind, we implemented three methods that rely on progressively less explicit knowledge of the environment.

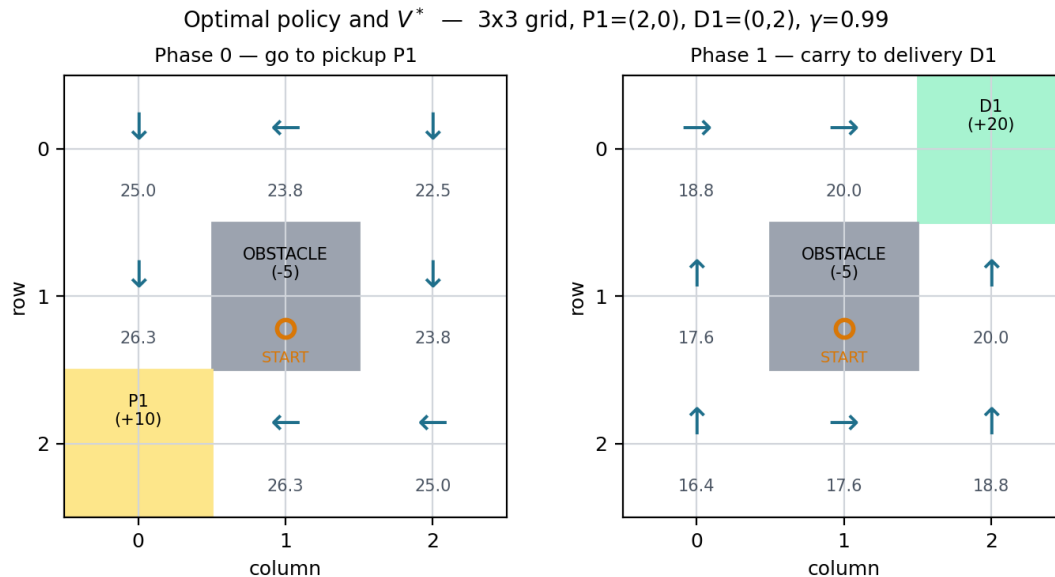
- Policy evaluation for a fixed “always move right” policy, using $\gamma = 0.99$ and a tolerance of 10^{-8} . The goal was to establish a baseline and confirm that the implementation reproduced the analytical value of a deliberately poor policy.
- Value iteration, using the same γ and tolerance to obtain V^* and π^* . This provides an exact optimal reference against which the learned policy can later be compared.
- Q-learning, trained for 3,000 episodes with $\alpha = 0.1$, $\gamma = 0.99$, and an ϵ – *greedy* policy. ϵ starts at 1.0, decays by a factor 0.995 after each episode, and is clipped at 0.05. Each episode is capped at 200 steps. The goal is to learn a policy from experience alone.

4. Establishing an optimal reference with Value Iteration

Before letting the agent learn, we need to know what a correct solution should look like.

Value iteration gives us that reference because it has access to the transition and reward functions. Without running a single episode, it computes the optimal value of every state. Figure 1 shows those values and the corresponding optimal policy for both phases of the task.

Figure 2. Optimal policy (arrows) and optimal state values V^ (numbers) computed by value iteration for both phases of the episode. The starting cell coincides with the obstacle.*



Value iteration converged in nine sweeps to an optimal value of 25.01 at the initial state. That number can be derived exactly. A rough estimate is easy: the two positive rewards sum to +30, and the route requires several costly moves, so the final value should be somewhere around 25.

Following the optimal policy:

Down (-1) → left (+10, pickup) → up (-1) → up (-1) → right (-1) → right (+20, delivery)

The episode takes six actions, but only four incur the -1 step cost because the pickup and delivery actions receive their task rewards instead. The undiscounted return is therefore $30 - 4 = 26$. Discounting the later rewards gives $25.0088 \approx 25.01$.

One distinction matters before moving on: 25.01 is the optimum for one fixed layout, with $P1 = (2,0)$ and $D1 = (0, 2)$. In the Q-learning experiments, $P1$ and $D1$ are randomized at the start of each episode, so the appropriate reference must change. We return to that distinction in the next section.

5. Q-Learning learns the task, then stops improving

Value iteration solves the task, but it does so with full knowledge of the environment. In a real delivery setting, an agent would not necessarily know the transition dynamics in advance. Q-learning does not require an explicit transition model; instead, it learns action values from sampled experience.

With the compact state representation, 3,000 training episodes, and the ϵ -greedy exploration schedule described earlier, Q-learning achieved an average return of 22.85 ± 0.36 over the final 100 episodes and completed the delivery in 100% of them. The learning curve in

Figure 4 shows three broad phases: roughly 200 episodes of mostly random behavior, a sharp improvement between episodes 200 and 500, and a plateau from that point onward.

One detail about the exploration schedule changes how this run should be interpreted. Starting from $\epsilon = 1.0$ and multiplying it by 0.995 after each episode, the lower bound of 0.05 is reached at episode 598. This means that more than 80% of the 3,000 episodes were run at the minimum exploration rate. In other words, the usual description that “the agent explores less and less over time” is only accurate for the first fifth of training; the rest is mostly exploitation with a small, constant amount of exploratory noise.

This is where the result that eventually drove the analysis appears. Unlike the fixed layout used for value iteration, the Q-learning experiments randomize P1 and D1 at the start of every episode. That introduces three different reference values that should not be mixed:

- **25.01** — the discounted optimal value V^* for the fixed layout $P1 = (2, 0)$, $D1 = (0, 2)$.
- **27.64** — the mean discounted V^* across all 56 valid layouts.
- **28.21** — the mean optimal undiscounted return across the same 56 layouts. This is the appropriate ceiling for Q-learning, because the reported episode return is accumulated without discounting.

Using the correct reference, the agent is not 2.5 points below the optimum; it is 5.36 points below the comparable ceiling of 28.21.

And that gap did not close. The reward curve remains essentially flat from around episode 500 to episode 3,000. For several sessions, 22.85 was simply a number that did not make sense to us. We tested several explanations, and when we eventually understood it, the problem had nothing to do with learning speed.

A flat learning curve does not necessarily mean the agent needs more training. Sometimes it means the learning problem itself has been defined incorrectly.

Written this way, the diagnosis may sound obvious in hindsight. It was not. We first suspected that the agent needed more episodes. Then, after asking an AI assistant for help, we tested the learning rate. We also considered the discount factor because the task has two stages and the plateau looked like it might be a horizon problem. None of those hypotheses explained the behavior.

6. Our first hypothesis: it needs more training

Our first explanation was simple: perhaps 3,000 episodes were not enough. We thought the Q-table might still be under-trained and that the plateau reflected insufficient experience rather than a limitation of the problem formulation.

The evidence did not strongly support that explanation. The reward curve rises sharply between roughly episodes 200 and 500 and then levels off. The remaining ~2,500 episodes produce no meaningful improvement, and ϵ reaches its minimum value of 0.05 at episode 598. From that point onward, most training consists of repeatedly exploiting the same policy with only 5% exploratory noise.

There was little evidence that a few thousand additional episodes would close the gap.

7. Second hypothesis: the learning rate

The second hypothesis came from the AI assistant. When we described the problem as “the agent plateaus below the optimum,” it suggested lowering the learning rate and training for more episodes. The recommendation was plausible, but in this case, it turned out to be wrong.

Even so, the experiment we designed to test it became the one that eventually revealed the real issue.

We evaluated five values of α under two conditions:

- **randomized targets**, where P1 and D1 were resampled at the start of each episode;
- **fixed targets**, where P1 and D1 remained in the same locations throughout training.

The result was the most surprising part of the project.

Figure 2. Average return over the final 100 episodes as a function of α , under randomized and fixed target positions. Mean $\pm$ standard deviation across ten seeds.

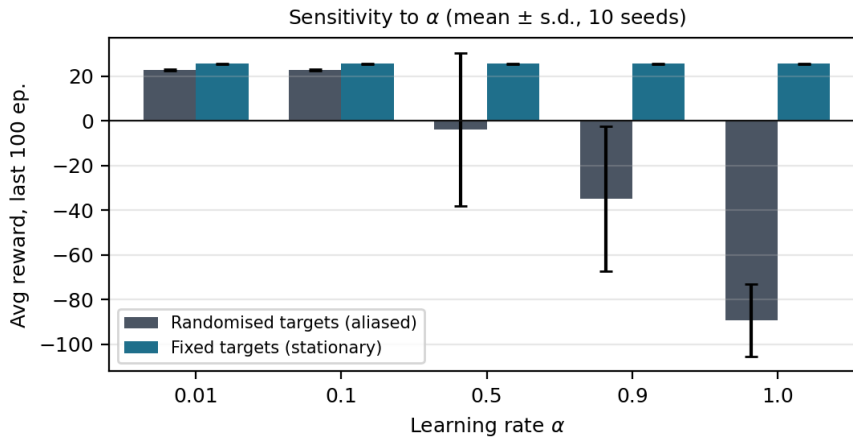


Table 2. Sensitivity to α . Ten seeds per condition; the fixed-target condition uses $P1 = (2, 0)$ and $D1 = (0, 2)$, for which the optimal undiscounted return is 26.

α	Randomized targets	Fixed targets
0.01	22.87 $\pm$ 0.34	25.54 $\pm$ 0.16
0.10	22.85 $\pm$ 0.36	25.51 $\pm$ 0.16
0.50	-3.82 $\pm$ 34.36	25.54 $\pm$ 0.17
0.90	-34.75 $\pm$ 32.51	25.57 $\pm$ 0.18
1.00	-89.11 $\pm$ 16.18	25.55 $\pm$ 0.17

In the fixed-target condition, α had very little effect across the range tested. Every value from 0.01 to 1.0 converged to the same narrow band, just below the undiscounted optimum of 26. Within this stationary setup, all five learning rates found essentially the same solution.

With randomized targets, α became critical. Large values collapsed: at $\alpha = 1.0$, the agent achieved an average return of about -89 , and the variability across seeds was substantial. The runs were not simply worse; they were qualitatively unstable.

The mechanism is consistent with state aliasing. Because the layout changes between episodes while the compact state remains the same, experience collected under one layout can contradict experience collected under another. A small α averages those contradictory updates into a stable compromise. A large α gives too much weight to the most recent layout, causing the policy to thrash between incompatible solutions.

This changed how we interpreted $\alpha = 0.1$. We had chosen it because gradual updates seemed safer. The experiments showed a more specific reason: a low learning rate stabilizes the compromise policy under aliasing. It does not solve the representational problem, but it is the difference between a stable return around 22.85 and a collapse near -89 .

8. The control experiment: fixed vs randomized targets

The previous experiment was not just a hyperparameter sweep. Its most important feature was the control condition.

We ran two otherwise identical setups. In the first, P1 and D1 were randomized at the start of every episode. In the second, the targets remained fixed throughout all 3,000 episodes. The fixed-target condition was not the problem we ultimately wanted to optimize; it was introduced as a control to isolate what changed when the targets stopped moving.

Considered separately, each column is easy to misread. The randomized condition suggests that “large α is bad,” which is true but superficial. The fixed condition reveals something more important: once the environment is stationary, α matters very little across the

range tested. Every value between 0.01 and 1.0 converges to nearly the same return, close to the optimum for that layout.

That result changed the diagnosis. If α were the main cause of the plateau, we would expect it to degrade performance in the fixed condition as well. It did not.

The control also made a general failure in the training loop much less plausible. Both conditions use the same Q-learning update, the same reward function, and the same training procedure. The main difference is whether P1 and D1 are resampled between episodes. Because the implementation performs near optimally when the targets are fixed, the explanation shifts toward what the two conditions do not share.

Seen side by side, the two columns tell us that the hyperparameter was not the core problem. The problem was that the task changed between episodes without that change being represented in the agent's state.

That was the point at which the investigation stopped being about tuning and became a question about observability.

The answer was **state aliasing**.

9. What 10 seeds changed about our conclusions

Each experiment changes one factor while holding the others fixed. Because Q-learning is stochastic, a single run can illustrate a behavior, but it cannot tell us how reliable that behavior is. Every configuration reported in this article was therefore run with ten different random seeds and is reported as the mean $\pm$ standard deviation of the average return over the final 100 episodes.

The factors we varied were the state representation, the learning rate α , and the discount factor γ .

With only one run per configuration, the discount-factor sweep could easily be read as “ γ does not matter much in this problem”. Across ten seeds, the picture changes. $\gamma = 0.95$ collapses in some runs, while $\gamma = 0.99$ produces both the highest average return and the most stable results.

The same pattern appears for $\alpha = 0.5$. It means return of -3.82 hides a standard deviation of 34.36 . A single run could have landed anywhere within that spread and supported either conclusion: that $\alpha = 0.5$ works reasonably well, or that it fails badly.

Repetition was not the only thing that mattered. Without the fixed-target control, the entire sweep would have ended as a hyperparameter recommendation rather than a diagnosis of the state representation. The seeds told us how much confidence to place in each number; the control told us where to look.

A result reported without variability is not simply less precise. In a stochastic system, it can be qualitatively misleading.

The conclusion does not merely become more precise; it becomes different.

10. The real problem: state aliasing

The agent plateaued at an average return of roughly 23, and additional training did not close the gap. The problem was not a hyperparameter. It was the state representation.

P1 and D1 are resampled at the start of every episode, but the agent observes only:

(row, column, phase)

The pickup and delivery locations are missing from the state. As a result, the agent can wake up in a differently configured city on every episode without being able to tell that anything has changed.

This is **state aliasing**: genuinely different situations are represented as the same state. The agent is therefore forced to learn one action-value estimate for situations that may require different decisions. What emerges is a compromise policy that performs reasonably well on average but cannot be optimal across all layouts.

Once the correct benchmark is used, the size of the problem becomes clearer. The compact agent achieves 22.85, compared with a mean optimal undiscounted return of 28.21 — a gap of 5.36 points, not the roughly 2.5-point gap suggested by comparing against the fixed-layout value of 25.01.

The notebook measures this limitation directly. We froze the learned policy from each of the ten seeds and evaluated it greedily, with no exploration, across all 56 valid layouts. The compact policy completed every layout but was optimal in only 8 of the 56. Its average loss relative to the optimum was 4.71 points, reaching 12 points in the worst case.

The full-state policy behaves very differently. Under the same evaluation procedure, it is optimal in an average of 53.4 out of 56 layouts across the ten seeds, with a range of 51–55 and has an average loss of only 0.11 points.

One detail is especially revealing: for the compact representation, the standard deviation of the greedy policy loss is exactly zero. All ten seeds converge to the same evaluated outcome — the same mean return, the same 8 optimal layouts, and the same worst case. That makes the remaining gap much more plausibly a systematic limitation of the representation than a consequence of an unlucky seed or noisy training.

11. Giving the agent the information it was missing

The appropriate fix is not to tune the learning rate again. It is to give the agent the information that is missing from its state.

We therefore extend the compact representation:

$$(row, column, phase)$$

to include the coordinates of both P1 and D1:

$$(row, column, P1_{row}, P1_{col}, D1_{row}, D1_{col}, phase)$$

This makes layouts that were previously aliased become distinct states.

Figure 3. Full-state representation including the agent position, P1, D1, and phase.

```
def get_full_state(env):  
    return (env.pos[0], env.pos[1],  
            env.p1[0], env.p1[1],  
            env.d1[0], env.d1[1],  
            env.phase)
```

Table 3. Effect of the state representation. Ten seeds, randomized targets, 3,000 episodes per run.

State representation	Visited state keys	Q-values ($\times 4$ actions)	Avg. return (final 100 episodes)
(row, col, phase)	18	72	22.85 ± 0.36
+ P1 and D1 coordinates	806 on average (seed 0: 820; range: 779–820)	$\approx 3\,224$	27.59 ± 0.12
Comparable ceiling — mean optimal undiscounted return, 56 layouts	—	—	28.21
Reference — mean discounted V^* , 56 layouts	—	—	27.64

The effect is immediate. The compact representation achieves an average training return of 22.85 ± 0.36 , whereas the full-state representation reaches 27.59 ± 0.12 .

The full-state result is only 0.62 points below the mean optimal undiscounted return of 28.21. That remaining difference can be examined more carefully by separating the learned greedy policy from the return observed during ϵ -greedy training.

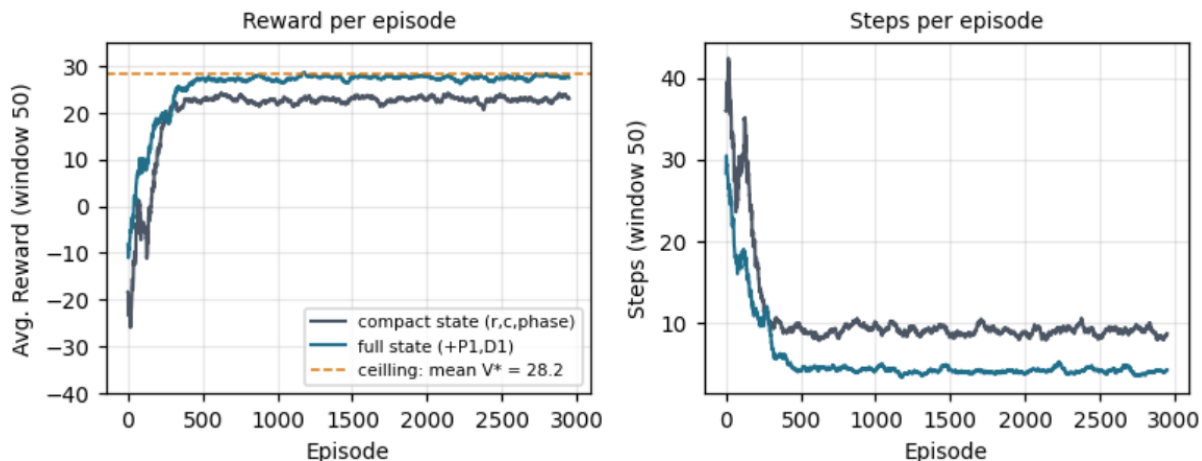
Table 4. Decomposition of the performance gap. Ten seeds; the frozen-policy row evaluates the learned policy greedily, with no exploration, across all 56 layouts.

	Compact state	Full state
Mean optimal return (undiscounted)	28.21	28.21
Learned policy, frozen (greedy)	23.50 ± 0.00	28.10 ± 0.06
Return during training ($\epsilon = 0.05$)	22.85 ± 0.36	27.59 ± 0.12
Policy loss (state aliasing)	4.71 ± 0.00	0.11 ± 0.06
Gap between greedy evaluation and training return	0.65 ± 0.36	0.52 ± 0.13

The frozen greedy evaluation makes the difference especially clear. The full-state policy reaches 28.10 ± 0.06 , compared with the mean optimal return of 28.21. In contrast, the compact policy remains at 23.50 ± 0.00 even after exploration is removed.

This means that almost all of the difference between the two agents lies in the learned policy itself. For full-state representation, the remaining gap between greedy evaluation and training return is small and largely consistent with the 5% residual exploration used during training.

Figure 4. Learning curves for the two state representations (single seed, 50-episode moving average). The compact-state agent plateaus below the optimal-return ceiling and stabilizes at roughly nine steps per delivery, whereas the full-state agent approaches the ceiling and stabilizes at around four steps.



Unless stated otherwise, all experiments in this article use a 3×3 grid with the obstacle at $(1, 1)$ and $\gamma = 0.99$. In the fixed-layout experiments — Figure 1 and the control condition — P1 is located at $(2, 0)$ and D1 at $(0, 2)$. In the remaining experiments, P1 and D1 are resampled at the start of every episode.

12. The trade-off: 18 states become 1008

The jump from 22.85 to 27.59 did not come from tuning another hyperparameter. It came from changing what the agent was able to observe.

That improvement has a cost. The compact representation contains only 18 possible states. Once the P1 and D1 coordinates are added, the full representation expands to 1,008 possible states — 56 times larger. During training, the agent visited about 806 of those state keys on average, roughly 45 times as many as in the compact representation.

At this scale, the increase is still manageable. On a realistic city map, it would not be.

Before looking at the larger example, it is useful to distinguish between states and Q-values. A Q-table stores one value for each state–action pair, not one value per state. With four possible actions, the number of Q-values is therefore four times the number of states.

For the 3×3 grid, the compact representation contains:

$$18 \text{ states} \times 4 \text{ actions} = 72 \text{ } Q\text{-values}$$

The full representation contains at most:

$$9 \text{ bicycle positions} \times 56 \text{ layouts} \times 2 \text{ phases} = 1,008 \text{ states}$$

or:

$$1,008 \times 4 = 4,032 \text{ } Q\text{-values.}$$

In practice, training visited an average of 806 state keys across the ten seeds, ranging from 779 to 820. That corresponds to roughly 3,224 stored Q -values on average.

The scaling problem becomes obvious on a 10×10 grid. A naive count gives:

$$100 \times 100 \times 100 \times 2 = 2,000,000 \text{ states}$$

or 8 million state–action values.

The environment rules eliminate some combinations because P1 and D1 cannot overlap each other, the starting cell, or the obstacle. That leaves 98 available cells and 9,506 valid ordered P1–D1 layouts:

$$100 \times 9,506 \times 2 = 1,901,200 \text{ states,}$$

corresponding to:

$$7,604,800 \text{ } Q\text{-values.}$$

The constraints reduce the naive count by less than 5%, so they do not change the order of magnitude.

Even under the unrealistic assumption that every transition visited a new state, 30,000 transitions would cover at most about 1.6% of this state space. Even if every episode reached the 200-step cap, coverage would still remain limited, and most visited states would be seen only once.

At that point, a lookup table stops being a practical representation. The natural next step is to replace it with a function that generalizes across similar states — for example, learning that “the pickup point is two cells south” should imply similar behavior regardless of the bicycle’s absolute position.

That is the trade-off: adding observability fixes the aliasing problem, but it also causes the tabular state space to grow combinatorially. This is what motivates function approximation and, at larger scales, deep reinforcement learning.

13. Secondary findings: gamma and the 1834 sweeps

Two results sit outside the main diagnosis but are still worth recording.

The discount factor γ

Because the task has two stages and the largest reward arrives at the end, we also tested whether the plateau could be caused by an overly short effective horizon. We evaluated γ under the randomized-target condition using the compact state representation.

Table 5. Sensitivity to γ . Randomized targets, compact state, 3,000 episodes per run.

γ	0.5	0.9	0.95	0.99
Avg. return (10 seeds)	22.74 ± 0.68	20.02 ± 5.34	15.90 ± 14.72	22.85 ± 0.36

The optimal route is only about six steps long. Even with $\gamma = 0.5$, a reward six steps away retains about 1.6% of its nominal value, which is still enough to distinguish among the available alternatives. More importantly, state aliasing already limits performance for reasons that are independent of γ .

The effect of γ became much clearer once each configuration was repeated across ten seeds. Intermediate values were noticeably less stable: $\gamma = 0.95$ showed a standard deviation of

14.7 because some runs collapsed, while $\gamma = 0.9$ performed several points below the best setting. In contrast, $\gamma = 0.99$ produced both the highest average return and the most stable results.

It is useful to separate two effects here. In Q-learning, γ affected performance and stability, but it did not explain the plateau: the best-performing value was the one we were already using, and the compact agent still stalled around 23. In policy evaluation, however, γ affected convergence speed rather than the final value.

Why policy evaluation took 1,834 sweeps

Evaluating the fixed “always move right” policy converged after 1,834 sweeps, with a final residual of 9.98×10^{-9} and an initial-state value of -100 . By comparison, value iteration converged in only nine sweeps.

The number of sweeps follows directly from how the residual contracts by roughly a factor of γ at each iteration:

$$\frac{\log(\epsilon)}{\log(\gamma)} = \frac{\log(10^{-8})}{\log(0.99)} \approx 1833$$

The implementation stopped at 1,834. With $\gamma = 0.9$, the same calculation would require only about 175 sweeps.

The contrast with the nine sweeps of value iteration is not an efficiency comparison between the two algorithms. The optimal policy reaches a terminal state after a finite number of steps, so its return can be propagated exactly once the terminal chain is resolved. The “always move right” policy never terminates: the bicycle eventually reaches the boundary and continues receiving -1 indefinitely. Its value is therefore the discounted infinite sum

$$-\frac{1}{1 - 0.99} = -100$$

and the 1,834 sweeps are the cost of shrinking that infinite tail below the convergence tolerance.

14. What AI helped with and what it didn't

We used Gemini during development mainly to debug Python code and to investigate outputs that did not initially make sense. We did not use it to define the MDP, choose the reward function, or formulate the main experimental hypotheses.

It was useful for mechanical problems. One real example was:

“My Q -table is a `defaultdict(lambda: np.zeros(4))` keyed on tuples. In one cell I build the key as `(row, col, phase)` and in another as a 7-element tuple. Would the lookup fail silently instead of raising? Show me what actually happens at runtime.”

The assistant was much less reliable on the question that mattered most. When asked why the agent plateaued below the optimum, it suggested lowering the learning rate and training for more episodes. That was a plausible hypothesis, but it was not the correct diagnosis in this case.

The state-aliasing explanation emerged from comparing the randomized- and fixed-target conditions — an experiment that had to be designed, not merely requested. Any derivation suggested by the tool was checked manually against the code and the observed outputs before being used in this article.

The practical lesson was simple: AI was useful for debugging and hypothesis generation, but a plausible explanation was not evidence. The experiment had to decide.

15. What I learned

- **State representation mattered more than hyperparameter tuning.** Adding P1 and D1 to the state increased the average training return from 22.85 ± 0.36 to 27.59 ± 0.12 . More

importantly, the compact-state gap remained even after exploration was removed, showing that the main limitation was structural rather than stochastic.

- **A control condition can change the diagnosis, not just strengthen the experiment.** The fixed-target condition turned a tuning question — “is α wrong?” — into a representation question: “what changes between episodes that the agent cannot observe?”
- **Stochastic results need variability, not isolated runs.** Repeating each configuration across ten seeds changed the interpretation of the γ sweep. $\gamma = 0.95$ could collapse on some runs, while $\gamma = 0.99$ was the most stable. Reporting only one run could have supported a qualitatively different conclusion.
- **Fixing observability with tabular states creates a scaling problem.** Adding target coordinates removed the aliasing but expanded the possible state space by $56\times$. On a 10×10 grid, the same representation already requires about 7.6 million Q-values, motivating function approximation and, at larger scales, deep reinforcement learning.

The broadest lesson is methodological. The numbers produced by the project were usually explainable, and the explanations were often more valuable than the numbers themselves. The 1,834 sweeps were not arbitrary; they followed from $\log(10^{-8})/\log(0.99)$. The difference between nine and 1,834 sweeps was not about algorithmic efficiency but about termination. And the gap between the compact and full-state agents was not noise; it exposed a flaw in how the learning problem had been represented.

Whenever a result could not be explained, there was still something about the system that we had not understood.